

## PROYECTO PRX

# Presentación y defensa del Sprint 1

## Enlaces URL y tareas en Notas

Documento de apoyo para presentar y defender el incremento desarrollado durante el Sprint 1. El alcance se concentra en agregar enlaces web y listas de tareas al módulo de Notas, con validación, permisos, persistencia y actualización desde la vista.

| Dato           | Detalle                                   |
|----------------|-------------------------------------------|
| Entrega        | Presentación y defensa del Sprint 1       |
| Vencimiento    | 23 de septiembre de 2026 a las 23:59      |
| Módulo         | Notas                                     |
| Incremento     | Enlaces URL y tareas persistentes         |
| Criterios      | Puntualidad Calidad SOLID Patrones        |
| Estado técnico | Frontend y backend compilan correctamente |

## Conclusión del Sprint

El objetivo del Sprint 1 se cumple: una nota puede guardar enlaces válidos, mostrar recursos externos, administrar tareas y conservar su estado después de recargar. La solución respeta la arquitectura existente y protege las operaciones mediante autenticación y permisos del repositorio.

# 1 Objetivo y alcance

**Objetivo** Convertir una nota en un elemento de seguimiento y consulta, no solamente en texto aislado.

**Valor entregado** El usuario relaciona documentación externa y registra actividades pendientes o completadas dentro de la misma nota.

## 1.1 Incluido en el Sprint 1

- Agregar varios enlaces URL a una nota.
- Normalizar direcciones sin protocolo y aceptar únicamente HTTP o HTTPS.
- Mostrar, abrir y eliminar enlaces relacionados.
- Agregar y eliminar tareas desde el detalle o la edición de una nota.
- Marcar y desmarcar tareas directamente desde la vista de la nota.
- Guardar tareas y enlaces en MySQL mediante Prisma.
- Aplicar permisos según propietario, membresía y visibilidad del repositorio.

## 1.2 Fuera del Sprint 1

Las imágenes y el editor gráfico de Notas corresponden al Sprint 2. Los enlaces y tareas de Bitácora corresponden al Sprint 3. Pueden mencionarse como continuidad del mantenimiento, pero no deben presentarse como parte del incremento evaluado en este Sprint 1.

# 2 Historias de usuario

| ID     | Historia                                                                                                | Puntos | Estado |
|--------|---------------------------------------------------------------------------------------------------------|--------|--------|
| HU M01 | Como usuario quiero agregar enlaces URL a una nota para relacionarla con recursos externos.             | 5      | Hecho  |
| HU M02 | Como usuario quiero administrar tareas en una nota para controlar actividades pendientes y completadas. | 8      | Hecho  |

## 2.1 Criterios de aceptación de enlaces

- Se puede escribir una dirección con o sin protocolo.
- Solo se aceptan enlaces HTTP o HTTPS válidos.
- Se pueden agregar varios enlaces y eliminar cualquiera antes de guardar.
- Los enlaces guardados aparecen en el detalle y se abren en una pestaña nueva.
- Los enlaces permanecen después de recargar la página.

## 2.2 Criterios de aceptación de tareas

- La tarea debe tener un título no vacío.
- Las tareas se pueden agregar y eliminar durante la edición.
- El detalle muestra las tareas pendientes y completadas.
- El checkbox permite cambiar el estado sin entrar a Editar.
- El estado se guarda inmediatamente y permanece después de recargar.

## 3 Sprint Backlog terminado

| Tarea | Trabajo técnico                       | Evidencia                     | Estado   |
|-------|---------------------------------------|-------------------------------|----------|
| T1    | Definir NoteTaskModel y NoteLinkModel | note model ts                 | Hecho    |
| T2    | Crear controles para URL              | create note y note detail     | Hecho    |
| T3    | Normalizar y validar HTTP o HTTPS     | normalizeUrl e isValidUrl     | Hecho    |
| T4    | Crear lista de tareas y checkbox      | note detail HTML y TS         | Hecho    |
| T5    | Enviar listas mediante FormData       | note api ts                   | Hecho    |
| T6    | Analizar y validar JSON en backend    | UpdateNoteHandler             | Hecho    |
| T7    | Persistir listas en MySQL             | schema y PrismaNoteRepository | Hecho    |
| T8    | Aplicar permisos del repositorio      | Handlers de Notes             | Hecho    |
| T9    | Verificar compilación                 | ng build y nest build         | Correcto |
| T10   | Preparar exposición                   | Documento y demostración      | Listo    |

## 4 Demostración funcional

1. Iniciar sesión y abrir una nota existente.
2. Entrar a Editar y agregar una URL sin protocolo, por ejemplo ejemplo.com.
3. Comprobar que el sistema la convierte en https://ejemplo.com.
4. Agregar una tarea con un título breve y guardar la nota.
5. Regresar al modo Ver y abrir el enlace en una pestaña nueva.
6. Marcar la tarea desde el checkbox sin entrar nuevamente a Editar.
7. Recargar la página y comprobar que el enlace y el estado permanecen.
8. Mostrar un intento con URL inválida para evidenciar la validación.

### Resultado que se debe señalar

La demostración prueba el flujo completo: interacción en Angular, envío al backend, validación, autorización, persistencia en MySQL y recuperación del dato confirmado. No es solamente una maqueta visual.

## 5 Código implementado

### 5.1 Contratos de dominio en Angular

```
export interface NoteTaskModel {
  id: string;
  title: string;
  completed: boolean;
}

export interface NoteLinkModel {
  id: string;
  url: string;
}
```

**Qué hace** Define la estructura de los datos que utilizan la pantalla, la API y el Store.

**Por qué se usó** TypeScript detecta errores de contrato antes de ejecutar la aplicación y facilita mantener las capas sincronizadas.

### 5.2 Validación y normalización de URL

```
private normalizeUrl(value: string): string {
  const trimmed = value.trim();
  if (!trimmed) return '';
  return /^https?:\/\//i.test(trimmed) ? trimmed : `https://${trimmed}`;
}

private isValidUrl(value: string): boolean {
  try {
    const url = new URL(value);
    return url.protocol === 'http:' || url.protocol === 'https:';
  } catch {
    return false;
  }
}
```

normalizeUrl mejora la experiencia al completar HTTPS cuando falta. isValidUrl utiliza el analizador URL del navegador y limita los protocolos. El backend vuelve a validar porque la interfaz nunca debe ser la única barrera.

### 5.3 Cambio de estado de tareas

```
protected toggleTask(task: NoteTaskModel): void {
  if (!this.canEdit() || this.saving()) return;
  this.tasks.update((tasks) => tasks.map((item) =>
    item.id === task.id ? { ...item, completed: !item.completed } : item
  ));
  if (!this.editing()) this.persist(false);
}
```

La señal se actualiza de forma inmutable. Si el usuario está en Ver, persist false guarda el cambio sin abrir el formulario de edición. saving evita ejecutar dos actualizaciones al mismo tiempo.

## 5.4 Transporte mediante FormData

```
formData.append('tasks', JSON.stringify(data.tasks));
formData.append('links', JSON.stringify(data.links));
formData.append('retainedFileIds', data.retainedFileIds.join(','));
```

Notas ya utiliza multipart para adjuntos. Por eso las listas se serializan como JSON dentro de FormData, manteniendo una sola petición compatible con texto y archivos.

## 5.5 Validación en el caso de uso

```
const tasks = this.parseTasks(command.dto.tasks);
const links = this.parseLinks(command.dto.links);

await this.noteRepository.update(command.id, {
  title: command.dto.title.trim(),
  content: command.dto.content.trim(),
  tasks,
  links,
  updatedBy: command.userId,
});
```

- parseTasks exige arreglo, limita a 100 tareas, valida id y título y recorta el título.
- parseLinks exige arreglo, limita a 50 enlaces, analiza cada URL y limita a 2048 caracteres.
- Antes de actualizar se comprueba que la nota exista y que el usuario tenga permiso.

## 5.6 Campos de base de datos

```
model Note {
  tasks Json?
  links Json?
}
```

Sí existen campos para guardar correctamente las dos listas. Prisma las escribe como JSON opcional y el Mapper devuelve arreglos vacíos cuando una nota antigua todavía no tiene esos valores.

# 6 Arquitectura y recorrido

| Capa                     | Elemento                    | Responsabilidad                               |
|--------------------------|-----------------------------|-----------------------------------------------|
| Presentación             | Note detail component       | Capturar acciones y mostrar tareas y enlaces. |
| Aplicación frontend      | NoteFacade                  | Coordinar API, loading y Store.               |
| Infraestructura frontend | NoteApi                     | Construir FormData y ejecutar HTTP.           |
| Presentación backend     | NotesController             | Recibir la petición y extraer el usuario JWT. |
| Aplicación backend       | UpdateNoteHandler           | Validar permisos y reglas del caso de uso.    |
| Dominio                  | NoteEntity y NoteRepository | Representar datos y contrato de persistencia. |

| Capa                    | Elemento                        | Responsabilidad                                    |
|-------------------------|---------------------------------|----------------------------------------------------|
| Infraestructura backend | PrismaNoteRepository            | Guardar y recuperar JSON en MySQL.                 |
| Conversión              | Prisma Mapper y Response Mapper | Traducir entre base de datos, dominio y respuesta. |

## Flujo completo

Usuario → Componente Angular → Facade → NoteApi → NotesController → CommandBus → UpdateNoteHandler → NoteRepository → Prisma → MySQL → Mapper → respuesta → Store → interfaz.

# 7 Calidad

## 7.1 Evidencia de calidad

| Evidencia        | Resultado   | Qué demuestra                                      |
|------------------|-------------|----------------------------------------------------|
| Build Angular    | Correcto    | El frontend genera el bundle sin errores.          |
| Build NestJS     | Correcto    | Prisma Client y TypeScript compilan correctamente. |
| Validación URL   | Dos capas   | Frontend mejora UX y backend protege los datos.    |
| Permisos         | Aplicados   | No se modifican notas sin propiedad o membresía.   |
| Persistencia     | Comprobable | Los datos sobreviven a una recarga.                |
| Enlaces externos | Protegidos  | Se usa noopener noreferer al abrir otra pestaña.   |
| Estado saving    | Aplicado    | Evita acciones duplicadas durante una petición.    |
| Borrado lógico   | Aplicado    | Se mantiene trazabilidad mediante status.          |

## 7.2 Definición de terminado

- La interfaz administra enlaces y tareas sin errores de maquetación.
- Frontend y backend compilan.
- Los datos se guardan y se recuperan mediante Prisma.
- Las entradas inválidas reciben mensajes comprensibles.
- Los permisos impiden modificar notas sin acceso al repositorio.
- La demostración puede repetirse antes de la entrega.

## 7.3 Mejoras pendientes

- Agregar pruebas automatizadas unitarias y de integración para las nuevas funciones.
- Permitir agregar tareas directamente durante la creación inicial de una nota; actualmente se administran desde detalle y edición.

- Reducir el SCSS de create note page, que supera el presupuesto Angular por 818 bytes.

## 8 Principios SOLID

| Principio                   | Aplicación en el Sprint 1                                                             | Defensa                                                            |
|-----------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| S Responsabilidad única     | Component, Facade, API, Handler, Repository y Mapper tienen tareas distintas.         | Los cambios quedan localizados en la capa responsable.             |
| O Abierto cerrado           | Contratos y componentes permiten ampliar comportamientos sin reescribir consumidores. | Se agregaron listas respetando la arquitectura existente.          |
| L Sustitución de Liskov     | PrismaNoteRepository cumple el contrato NoteRepository.                               | El caso de uso puede trabajar con cualquier implementación válida. |
| I Segregación de interfaces | NoteApiContract contiene operaciones específicas de Notas.                            | El módulo no depende de métodos ajenos a su necesidad.             |
| D Inversión de dependencias | UpdateNoteHandler inyecta NoteRepository abstracto.                                   | La lógica de aplicación no depende directamente de Prisma.         |

## Cómo explicarlo oralmente

No colocamos toda la lógica en la pantalla. El componente maneja interacción; la Facade coordina; la API transporta; el Handler aplica reglas y permisos; el Repository persiste; el Mapper transforma datos. Esta división es la evidencia principal de responsabilidad única e inversión de dependencias.

## 9 Patrones de diseño

| Patrón            | Uso en el Sprint 1                                        | Por qué se usó                                         |
|-------------------|-----------------------------------------------------------|--------------------------------------------------------|
| CQRS              | UpdateNoteCommand para escribir y Queries para consultar. | Separar lecturas y cambios en casos de uso pequeños.   |
| Repository        | NoteRepository y PrismaNoteRepository.                    | Desacoplar negocio de Prisma y MySQL.                  |
| Facade            | NoteFacade entre las páginas, API y Store.                | Simplificar el componente y centralizar coordinación.  |
| Mapper            | NotePrismaMapper y NoteResponseMapper.                    | Evitar filtrar formatos de persistencia a la interfaz. |
| DTO               | UpdateNoteRequestDto.                                     | Definir y validar la entrada HTTP.                     |
| Reactive Store    | Signals y NoteStore.                                      | Mantener estado predecible de datos, carga y error.    |
| Guard y Decorator | JWT y CurrentUser.                                        | Centralizar identidad y autorización.                  |
| Soft delete       | status igual a cero.                                      | Conservar auditoría sin eliminar físicamente.          |

## Cómo defender los patrones

Los patrones no se usaron solo para nombrarlos. CQRS organiza operaciones, Repository aísla la base de datos, Facade reduce dependencias de la UI, Mapper protege límites entre capas y DTO impide que datos incorrectos lleguen al caso de uso. Cada patrón responde a un problema concreto del Sprint.

## 10 Puntualidad

La entrega vence el 23 de septiembre de 2026 a las 23:59. Para demostrar puntualidad se debe enviar el trabajo antes de esa hora y conservar evidencia del envío. El código y el documento están preparados; la confirmación final en la plataforma corresponde al estudiante.

| Evidencia            | Estado                   | Acción antes de entregar          |
|----------------------|--------------------------|-----------------------------------|
| Historias del Sprint | Hechas                   | Demostrar ambos flujos.           |
| Frontend             | Compila                  | Conservar salida de ng build.     |
| Backend              | Compila                  | Conservar salida de nest build.   |
| Documento            | Preparado                | Adjuntarlo con la entrega.        |
| Registro de envío    | Pendiente del estudiante | Guardar captura con fecha y hora. |

## 11 Guion de exposición

| Tiempo | Tema         | Qué decir o mostrar                                     |
|--------|--------------|---------------------------------------------------------|
| 1 min  | Objetivo     | Explicar el problema y el alcance real del Sprint 1.    |
| 3 min  | Demostración | URL, tarea, guardado, checkbox y recarga.               |
| 2 min  | Código       | Modelo, validación, FormData, Handler y Prisma.         |
| 2 min  | Calidad      | Builds, validación en dos capas, permisos y límites.    |
| 2 min  | SOLID        | SRP y DIP con la separación de capas.                   |
| 2 min  | Patrones     | CQRS, Repository, Facade, Mapper y DTO.                 |
| 1 min  | Cierre       | Valor entregado, alcance cumplido y siguientes sprints. |

### 11.1 Mensaje de apertura

En el Sprint 1 trabajamos el módulo de Notas. El objetivo fue permitir que una nota relacionara recursos web y administrara tareas pendientes o completadas. La solución abarca interfaz, validación, permisos, backend y persistencia.



## 11.2 Mensaje de cierre

El incremento cumple las dos historias comprometidas. Los enlaces y tareas se guardan, se recuperan y respetan los permisos del repositorio. La arquitectura separada permite continuar en el Sprint 2 con imágenes y en el Sprint 3 con Bitácora sin concentrar toda la lógica en un solo componente.

## 12 Preguntas probables

### ¿Qué entregaron exactamente?

Enlaces URL y tareas persistentes en Notas, incluyendo vista, edición, validación y permisos.

### ¿Dónde se guardan?

En los campos JSON opcionales tasks y links del modelo Note en MySQL mediante Prisma.

### ¿Cómo validan una URL?

Angular normaliza y utiliza new URL; el backend vuelve a analizar el valor antes de guardarlo.

### ¿Por qué usaron JSON?

Las listas son pequeñas, pertenecen a una sola nota y se recuperan junto con ella. Si luego necesitan consultas globales, podrían normalizarse en tablas.

### ¿Cómo evitan editar notas ajenas?

El JWT entrega el userId y el Handler comprueba propiedad o membresía según la visibilidad del repositorio.

### ¿Qué pasa al marcar una tarea desde Ver?

Signals actualiza la lista y persist false ejecuta PUT sin abrir el editor. El cambio queda guardado.

### ¿Dónde está SOLID?

En la separación Component, Facade, API, Handler, Repository y Mapper, y en la dependencia del Handler hacia un contrato abstracto.

### ¿Qué patrones usaron?

CQRS, Repository, Facade, Mapper, DTO, Store reactivo, Guards y soft delete.

### ¿Cómo demuestran calidad?

Con compilaciones correctas, validación en dos capas, permisos, límites y persistencia comprobable.

### ¿Qué falta mejorar?

Pruebas automatizadas, tareas en la creación inicial y reducir el SCSS que supera el presupuesto.

## 13 Lista final antes de enviar

- Abrir el documento y revisar nombre, fecha y alcance.
- Ensayar la demostración con una nota de prueba.
- Tener frontend, backend y base de datos listos antes de presentar.
- No incluir imágenes ni Bitácora como trabajo del Sprint 1.
- Enviar antes de las 23:59 y conservar captura del comprobante.